

Laboratorio de Programación Funcional 2026

FWhile

La tarea consiste en la implementación de un intérprete de un lenguaje imperativo que llamaremos FWhile. FWhile permite definir funciones y estructuras de control de flujo, como iteraciones y condicionales, mediante una sintaxis simple. El intérprete deberá realizar las tareas de chequeo de nombres y tipos, *pretty-printing* y evaluación de expresiones.

1 FWhile

La siguiente EBNF describe la sintaxis de FWhile, donde $\langle ident \rangle$ es un identificador válido y $\langle nat \rangle$ un número natural:

```
 $\langle program \rangle$  ::= {  $\langle fun \rangle$  }  
 $\langle fun \rangle$  ::= 'fun'  $\langle ident \rangle$   $\langle ident \rangle$   $\langle stmts \rangle$   $\langle expr \rangle$  ';'   
 $\langle stmts \rangle$  ::= '{' {  $\langle stmt \rangle$  ';' } '}'   
 $\langle stmt \rangle$  ::=  $\langle ident \rangle$  ':'  $\langle expr \rangle$   
          | 'while'  $\langle expr \rangle$   $\langle stmts \rangle$   
          | 'if'  $\langle expr \rangle$  'then'  $\langle stmts \rangle$  'else'  $\langle stmts \rangle$   
          | 'case'  $\langle expr \rangle$  'of'  $\langle clauses \rangle$   
 $\langle clauses \rangle$  ::= '{' {  $\langle clause \rangle$  ';' } '}'   
 $\langle clause \rangle$  ::=  $\langle pat \rangle$  '->'  $\langle stmts \rangle$   
 $\langle pat \rangle$  ::= 'Nil' | 'Cons'  $\langle pat \rangle$   $\langle pat \rangle$  | '['  $\langle nat \rangle$  |  $\langle blit \rangle$  |  $\langle ident \rangle$  | 'C'  $\langle pat \rangle$  ')'   
 $\langle expr \rangle$  ::=  $\langle ident \rangle$  |  $\langle nat \rangle$  |  $\langle blit \rangle$  | 'Cons'  $\langle expr \rangle$   $\langle expr \rangle$  | 'Nil'  
          | 'head'  $\langle expr \rangle$  | 'tail'  $\langle expr \rangle$  |  $\langle ident \rangle$   $\langle expr \rangle$   
          |  $\langle expr \rangle$   $\langle bop \rangle$   $\langle expr \rangle$  |  $\langle uop \rangle$   $\langle expr \rangle$  | 'C'  $\langle expr \rangle$  ')'   
 $\langle bop \rangle$  ::= '+' | '-' | '*' | '/' | '%' | '|' | '&&' | '==' | '<'   
 $\langle uop \rangle$  ::= '!' | '-'   
 $\langle blit \rangle$  ::= 'True' | 'False'
```

```

fun cuad xs {
    x := head xs;
    } Cons (x * x) Nil;

fun suma xs {
    sum := 0;
    while !(xs == Nil) {
        x := head xs;
        xs := tail xs;
        sum := sum + x;
    };
    } Cons sum Nil;

```

Figure 1: Programa con dos funciones

Un programa consiste en una secuencia (posiblemente vacía) de declaraciones de funciones. Cada función se define mediante la palabra clave **fun**, seguida de dos identificadores (el nombre de la función y su parámetro), un bloque de instrucciones y una expresión de retorno, terminando con **;**.

Un bloque de instrucciones (**stmts**) está delimitado por llaves y contiene una secuencia de instrucciones separadas por **;**. Las instrucciones pueden ser asignaciones (**:=**), iteraciones (**while**), condicionales (**if - then - else**) o estructuras de selección (**case - of**).

Las estructuras **case - of** utilizan cláusulas compuestas por un patrón y un bloque de instrucciones asociado. Los patrones pueden ser la lista vacía (**Nil**), patrones contruidos con **Cons**, literales enteros, literales booleanos o variables.

Las expresiones pueden ser variables, literales enteros, literales booleanos, listas contruidas mediante **Cons** y **Nil**, aplicaciones de las funciones destructoras de listas (**head** y **tail**), aplicaciones de funciones definidas por el usuario, aplicaciones de operadores binarios sobre dos subexpresiones o aplicaciones de operadores unarios sobre una subexpresión.

Los operadores binarios incluyen los operadores aritméticos **+**, **-**, *****, **/** y **%**, los operadores lógicos **||** y **&&**, y los operadores de comparación **==** y **<**. Los operadores unarios son la negación aritmética **-** y el no lógico **!**.

El lenguaje cuenta únicamente con tres tipos: enteros, booleanos y listas de enteros. Las funciones reciben y retornan listas de enteros, mientras que el tipo de las variables se infiere a partir de las expresiones que se les asignan.

En la Figura 1 podemos ver un ejemplo de programa, compuesto de dos funciones: **cuad**, que calcula el cuadrado del primer elemento de una lista y **suma**, que suma todos los elementos de una lista. Notar que siempre hay que retornar una lista, aunque tenga un solo elemento.

En la Figura 2 se definen dos funciones, **max** para calcular el máximo de dos enteros y **maximum** para calcular el máximo de una lista no vacía. La segunda función llama a la primera. Notar cómo se utiliza la lista en **max** para pasar más de un argumento.

```

fun max xs {
  m := 0;
  case xs of
    { Cons x (Cons y Nil) -> { if y < x then { m := x; }
                               else { m := y; }; };
    }
  } Cons m Nil;

fun maximum xs {
  m := head xs;
  xs := tail xs;
  while !(xs == Nil) {
    m := head (max (Cons m (Cons (head xs) Nil)));
    xs := tail xs;
  };
  } Cons m Nil;

```

Figure 2: Máximo de una lista

El módulo AST contiene un tipo algebraico de datos `Prog` que representa a los árboles de sintaxis abstracta de programas `FWhile` y el módulo `Parser` una función de parsing, que dada una cadena de caracteres con un programa `FWhile` retorna su árbol de sintaxis abstracta o los errores de sintaxis encontrados al intentar parsear:

```
parseProg :: String -> Either ParseError Prog
```

2 Pretty-Printing

Dado un árbol de sintaxis abstracta del programa, la función `ppProg` del módulo `PP`, que debe ser implementada como parte de la tarea, retorna un `String` con el pretty-printing del programa:

```
ppProg :: Prog -> String
```

El pretty-printing sigue las siguientes reglas de formato:

- Cada definición de función comienza en una nueva línea.
- La llave de apertura de una función, un `while`, un `if - then - else` o un `case - of` se imprime al final de la línea que introduce el bloque.
- El contenido de un bloque se imprime con una indentación adicional de cuatro espacios. La llave de cierre se alinea con el comienzo de la construcción que abrió el bloque.
- Cada instrucción se imprime en una línea independiente.

- En un **case**, cada cláusula se imprime en una línea independiente y su bloque asociado se formatea siguiendo las reglas generales para bloques.
- Toda aplicación (de función, operador o de **Cons**) se imprime entre paréntesis.
- Entre dos elementos léxicos consecutivos se imprime exactamente un espacio, excepto junto a paréntesis, llaves y puntos y coma.
- No deben imprimirse espacios en blanco al final de las líneas.

Notar que las cláusulas de un **case** se consideran elementos de un bloque y sus cuerpos se consideran bloques anidados. La Figura 3 muestra el resultado de hacer pretty-printing a las funciones de los ejemplos de las figuras 1 y 2.

3 Chequeos

El intérprete debe realizar chequeos de nombres y de tipos, como se describe a continuación. Estos chequeos deben ser realizados por la función `checkProg` del módulo `TypeChecker`, que debe ser implementada como parte de la tarea:

```
data CheckRes = Ok
               | HasNameErrors [NameError]
               | HasTypeErrors [TypeError]
```

```
checkProg :: Prog -> CheckRes
```

La función toma como entrada el árbol de sintaxis abstracta de un programa y retorna `Ok` si el programa es correcto o, en otro caso, la lista de errores encontrados. Si se encuentran errores de nombres no se realiza el chequeo de tipos.

También se debe implementar la función `checkExp`, que utilizará el intérprete para chequear que una expresión a ser evaluada sólo refiere a funciones definidas en el programa y que no tiene errores de tipos:

```
checkExp :: Prog -> Exp -> CheckRes
```

3.1 Chequeo de Nombres

La primera etapa de chequeos consiste en el chequeo de nombres. Se debe verificar que no se usen nombres que no se hayan declarado anteriormente y que no hayan múltiples declaraciones de una misma función. Las variables se declaran la primera vez que se asignan y siguen las reglas de alcance usuales, es decir que son globales a los bloques internos de donde fueron declaradas y no son visibles en bloques externos. Un patrón no puede introducir variables que ya hayan sido declaradas y una variable no puede aparecer más de una vez en un mismo patrón.

Por ejemplo, en el programa de la Figura 4 se detectan los siguientes errores de nombres:

```
fun cuad xs {
  x := (head xs);
} (Cons (x * x) Nil);

fun suma xs {
  sum := 0;
  while (!(xs == Nil)) {
    x := (head xs);
    xs := (tail xs);
    sum := (sum + x);
  };
} (Cons sum Nil);

fun max xs {
  m := 0;
  case xs of {
    (Cons x (Cons y Nil)) -> {
      if (y < x) then {
        m := x;
      } else {
        m := y;
      };
    };
  };
} (Cons m Nil);

fun maximum xs {
  m := (head xs);
  xs := (tail xs);
  while (!(xs == Nil)) {
    m := (head (max (Cons m (Cons (head xs) Nil))));
    xs := (tail xs);
  };
} (Cons m Nil);
```

Figure 3: Ejemplo de Pretty-printing

```

fun foo xs {
    x := baz xs + tail ys + bar zs;
} ws;

fun bar ys {
    x := y + head xs + foo ys;
    x := 2;
    case ys of {
        (Cons w (Cons w ys) ) -> { x := w; };
    };
    xs := Cons x Nil;
    if x < 10 then { z := 0; } else { z := x;};
    zs := bar xs;
    x := z;
} zs;

fun foo xs {
} xs;

```

Figure 4: Chequeo de Nombres

```

undefined function: baz
undefined variable: ys
undefined function: bar
undefined variable: zs
undefined variable: ws
undefined variable: y
undefined variable: xs
duplicated variable: w
duplicated variable: ys
undefined function: bar
undefined variable: z
duplicated function: foo

```

Notar que el lenguaje no admite definiciones recursivas ni el uso de funciones o variables definidas posteriormente. Las repeticiones y el uso de nombres no definidos se listan a medida que se van encontrando en una recorrida de arriba hacia abajo. Las expresiones se recorren de izquierda a derecha.

3.2 Chequeo de Tipos

En caso de no existir errores en la etapa de chequeo de nombres, se procede con el chequeo de tipos. Los tipos del lenguajes son: `int` (enteros), `bool` (booleanos) y `list` (lista de enteros). Se debe verificar que los tipos son correctos de acuerdo a las siguientes reglas:

- El tipo del parámetro de una función y de su expresión de retorno es `list`.
- En las instrucciones `if` y `while` la condición es de tipo `bool`.

- En la asignación, el tipo de una variable que se asigna por primera vez es el de la expresión. En las siguientes asignaciones el tipo de la expresión debe coincidir con el tipo de la variable.
- Los tipos de los patrones de un **case** tienen que coincidir con el tipo de la expresión.
- El argumento de **head** y **tail** tiene tipo `list`. El resultado de **head** tiene tipo `int` y el de **tail** tiene tipo `list`.
- El argumento de una llamada a función definida por el usuario tiene tipo `list`. El resultado tiene tipo `list`.
- Las expresiones **Cons** y **Nil** tienen tipo `list`. Los argumentos de **Cons** tienen tipo `int` y `list`, respectivamente.
- Las expresiones compuestas por un operador aritmético (+, -, *, / y %) tienen tipo `int` y sus operandos también.
- Las expresiones compuestas por un operador lógico (!, || y &&) tienen tipo `bool` y sus operandos también.
- Las expresiones compuestas por un operador de comparación (== y <) tienen tipo `bool`. El operador == admite dos operandos del mismo tipo, mientras que < admite únicamente operandos de tipo `int`.
- Los literales enteros tienen tipo `int` y los literales booleanos tienen tipo `bool`. Las variables introducidas mediante asignación toman el tipo de su primera asignación. Las variables introducidas por patrones toman el tipo determinado por el patrón.
- Los patrones tienen las mismas reglas de tipado que sus expresiones correspondientes, a excepción del patrón variable, que puede tomar cualquier tipo, pero luego ese tipo queda fijo cuando se lo use como variable en las instrucciones correspondientes.

Los errores de tipos se listan a medida que se van encontrando en una recorrida de arriba hacia abajo. Los errores en las expresiones se listan en post-orden, es decir primero los errores de las sub-expresiones (de izquierda a derecha) y luego el error de la expresión que las compone.

Por ejemplo, en la Figura 5 se muestra un programa que generaría los siguientes errores:

```
invalid argument type in foo: int
invalid argument type/s in operator Add: list, int
invalid argument type/s in Cons: bool, list
invalid assignment in x: expected int, actual list
invalid argument type/s in operator Add: int, bool
invalid argument type/s in operator Or: bool, int
invalid argument type/s in operator And: int, bool
```

```

fun foo xs {
  } xs;

fun bar xs {
  x := head (foo 9);
  x := xs + x;
  x := Cons True Nil;
  x := (1 + True) && (False || 2);
  case xs of {
    9 -> { x := 9; };
    Cons Nil y -> { x := y; };
  };
  xs := True;
} x;

```

Figure 5: Programa con Errores de Tipos

```

invalid assignment in x: expected int, actual bool
invalid pattern: expected list, actual int
invalid argument type/s in Cons: list, list
invalid assignment in x: expected int, actual list
invalid assignment in xs: expected list, actual bool
invalid return type in bar: int

```

Notar que en los casos de tipos de argumentos incorrectos para **Cons** u operador binario, aunque uno de los tipos sea correctos igual se listan ambos.

4 Evaluación

Dados un programa y una expresión que se asumen chequeados, se debe definir una función de evaluación de la expresión en el contexto del programa. Esto es, que puede utilizar las funciones definidas en el programa. La evaluación se debe implementar por la función de `evalExp` del módulo `Eval`:

```
data RuntimeError = HeadOfEmptyList | TailOfEmptyList | DivisionByZero
```

```
type EvalRes = Either RuntimeError Val
```

```
evalExp :: Prog -> Exp -> EvalRes
```

Notar que la evaluación puede generar errores en tiempo de ejecución, que son: intentar acceder a la cabeza o cola de una lista vacía o dividir por cero. Como el programa y la expresión se asumen chequeados, no existen errores como variable no definida, función inexistente o error de tipos.

El resultado de la evaluación es un valor representado mediante el tipo `Val`. La evaluación sigue la semántica habitual de un lenguaje imperativo: las in-

strucciones se ejecutan secuencialmente modificando el estado de las variables, las iteraciones **while** repiten su cuerpo mientras la condición evalúe a verdadero, las expresiones **if** y **case** seleccionan la rama correspondiente según el valor de la condición o del patrón, y las llamadas a funciones utilizan las definiciones presentes en el programa.

Por ejemplo, dado el programa de la Figura 1:

- La expresión `cuad (Cons 8 Nil)` evalúa a `(Cons 64 Nil)`.
- La expresión `1 + head (cuad (Cons 8 Nil))` evalúa a 65.
- La expresión `suma (Cons 1 (Cons 2 (Cons 3 Nil)))` evalúa a `(Cons 6 Nil)`.
- La expresión `cuad Nil` evalúa al error de runtime `HeadOfEmptyList`.

Mientras que dado el programa de la Figura 2:

- La expresión `max (Cons 2 (Cons 5 Nil))` evalúa a `(Cons 5 Nil)`.
- La expresión `max (Cons 5 Nil)` evalúa a `(Cons 0 Nil)`.
- La expresión `maximum (Cons 1 (Cons 10 (Cons 8 (Cons 2 Nil))))` evalúa a `(Cons 10 Nil)`.
- La expresión `maximum Nil` evalúa al error de runtime `HeadOfEmptyList`.

5 Se Pide

Además de esta letra el obligatorio contiene los siguientes archivos:

`AST.hs` Módulo que contiene el AST.

`Parser.hs` Módulo que contiene el parser.

`TypeChecker.hs` Módulo de chequeos.

`PP.hs` Módulo de pretty-printing.

`Eval.hs` Módulo de evaluación.

`State.hs` Módulo que contiene un tipo y funciones para representar el estado de las variables en la evaluación.

`FWhile.hs` Programa Principal, importa los módulos anteriores y define un intérprete, que dado el *nombre* de un programa `FWhile` obtiene el programa de un archivo *nombre.fw*, chequea que sea válido y en caso de serlo, lo carga en un REPL para evaluar expresiones. En otro caso imprime los errores encontrados. El intérprete cuenta con una flag `-p` para desplegar el pretty-printing del programa.

La tarea consiste en modificar los archivos `TypeChecker.hs`, `PP.hs` y `Eval.hs`, implementando las funciones solicitadas, de manera que el intérprete se comporte como se describe en esta letra. En caso de que algún aspecto no quede completamente especificado en esta letra, será aclarado en el EVA del curso y/o en los ejemplos que se publicarán.

Los únicos archivos que se entregarán son `TypeChecker.hs`, `PP.hs` y `Eval.hs`. Dentro de ellos se pueden definir todas las funciones auxiliares que sean necesarias. No modificar ninguno de los demás archivos, dado que los mismos no serán entregados. Tampoco se deben modificar las definiciones de tipos ni las firmas de las funciones definidas en los archivos a entregar.

La tarea debe compilar usando el siguiente comando, de manera de asegurar que solo se utilizan los paquetes `base` y `parsec`:

```
ghc -hide-all-packages -package base -package parsec FWhile.hs
```

Esta tarea se encuentra en la **Categoría 1** de uso de IA definido por la “Guía para el uso ético y crítico de inteligencia artificial en las unidades curriculares de Facultad de Ingeniería”¹. Es decir que no se permite el uso de IA para “generar parte o toda la solución”. Cualquier trabajo entregable elaborado con herramientas de IA se considerará plagiado.

¹https://www.fing.edu.uy/sites/default/files/2026-02/guia-de-etica-fing_2026.pdf